

Creating NumPy array

In [1]: `import numpy as np`

In [2]: `np.array([2, 4, 56, 422, 32, 1])` *# 1D array*

Out[2]: `array([ 2, 4, 56, 422, 32, 1])`

In [3]: `a = np.array([2, 4, 56, 422, 32, 1])`
`print(a)`

`[ 2 4 56 422 32 1]`

In [4]: `type(a)`

Out[4]: `numpy.ndarray`

In [5]: *# 2D Array (Matrix)*

`new = np.array([[45, 34, 22, 2],[24, 55, 3, 22]])`
`print(new)`

`[[45 34 22 2]`
`[24 55 3 22]]`

In [6]: *# 3D ----- # Tensor - Collection of Matrixes*

`np.array([[2, 3, 33, 4, 45],[23, 45, 56, 66, 2],[357, 523, 32, 24, 2],[32, 32, 4`

Out[6]: `array([[ 2, 3, 33, 4, 45],`
`[ 23, 45, 56, 66, 2],`
`[357, 523, 32, 24, 2],`
`[ 32, 32, 44, 33, 234]])`

dtype

In [7]: `np.array([11, 23, 44], dtype = float)`

Out[7]: `array([11., 23., 44.])`

In [8]: `np.array([11, 23, 44], dtype = bool)` *# Here, True because , python treats Non-z*

Out[8]: `array([ True, True, True])`

In [9]: `np.array([11, 23, 44], dtype = complex)`

Out[9]: `array([11.+0.j, 23.+0.j, 44.+0.j])`

arange

In [10]: `np.arange(1,25)` *# 1-included, 25 - Last one got excluded*

```
Out[10]: array([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16, 17,
                18, 19, 20, 21, 22, 23, 24])
```

```
In [11]: np.arange(1,25,2) # Strides ---> Alternate numbers
```

```
Out[11]: array([ 1,  3,  5,  7,  9, 11, 13, 15, 17, 19, 21, 23])
```

reshape

```
In [12]: np.arange(1, 11).reshape(5,2) # Converted 5 rows and 2 columns
```

```
Out[12]: array([[ 1,  2],
                [ 3,  4],
                [ 5,  6],
                [ 7,  8],
                [ 9, 10]])
```

```
In [13]: np.arange(1,11).reshape(2,5) # Converted 2 rows and 5 columns
```

```
Out[13]: array([[ 1,  2,  3,  4,  5],
                [ 6,  7,  8,  9, 10]])
```

```
In [14]: np.arange(1,13).reshape(3,4) # Converted 3 rows and 4 columns
```

```
Out[14]: array([[ 1,  2,  3,  4],
                [ 5,  6,  7,  8],
                [ 9, 10, 11, 12]])
```

ones & zeros

```
In [15]: # np.ones and np.zeros
```

```
np.ones((3,4)) # We have to mention the tuple
```

```
Out[15]: array([[1., 1., 1., 1.],
                [1., 1., 1., 1.],
                [1., 1., 1., 1.]])
```

```
In [16]: np.zeros((3,4))
```

```
Out[16]: array([[0., 0., 0., 0.],
                [0., 0., 0., 0.],
                [0., 0., 0., 0.]])
```

```
In [17]: # Another Type ----> random()
```

```
np.random.random((4,3))
```

```
Out[17]: array([[0.65744122, 0.36285678, 0.49517462],
                [0.24571614, 0.34619275, 0.53491795],
                [0.64526217, 0.22397462, 0.82411256],
                [0.45337804, 0.45280204, 0.25372748]])
```

linspace

```
In [18]: np.linspace(-10, 10, 10) # Here: Lower range, upper range, number of item to g
```

```
Out[18]: array([-10.          , -7.77777778, -5.55555556, -3.33333333,
        -1.11111111,  1.11111111,  3.33333333,  5.55555556,
         7.77777778, 10.          ])
```

```
In [19]: np.linspace(-2, 12, 6)
```

```
Out[19]: array([-2. ,  0.8,  3.6,  6.4,  9.2, 12. ])
```

identity

```
In [20]: # Creating the identity matrix
```

```
np.identity(3)
```

```
Out[20]: array([[1., 0., 0.],
        [0., 1., 0.],
        [0., 0., 1.]])
```

```
In [21]: np.identity(6)
```

```
Out[21]: array([[1., 0., 0., 0., 0., 0.],
        [0., 1., 0., 0., 0., 0.],
        [0., 0., 1., 0., 0., 0.],
        [0., 0., 0., 1., 0., 0.],
        [0., 0., 0., 0., 1., 0.],
        [0., 0., 0., 0., 0., 1.]])
```

Array Attributes

```
In [22]: a1 = np.arange(10) # 1D
a1
```

```
Out[22]: array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
In [23]: a2 = np.arange(12, dtype = float).reshape(3,4) # Matrix
a2
```

```
Out[23]: array([[ 0.,  1.,  2.,  3.],
        [ 4.,  5.,  6.,  7.],
        [ 8.,  9., 10., 11.]])
```

```
In [24]: a3 = np.arange(8).reshape(2,2,2) # 3D ---> Tensor
a3
```

```
Out[24]: array([[[0, 1],
        [2, 3]],

        [[4, 5],
        [6, 7]]])
```

ndim

```
In [25]: a1.ndim
```

```
Out[25]: 1
```

```
In [26]: a2.ndim
```

```
Out[26]: 2
```

```
In [27]: a3.ndim
```

```
Out[27]: 3
```

shape

```
In [28]: a1.shape    # 1D array has 10 Items
```

```
Out[28]: (10,)
```

```
In [29]: a2.shape    # 3 rows and 4 columns
```

```
Out[29]: (3, 4)
```

```
In [30]: a3.shape    # first, 2 says it consists of 2D arrays. 2,2 gives no. of rows and c
```

```
Out[30]: (2, 2, 2)
```

size

```
In [31]: a3
```

```
Out[31]: array([[0, 1],
               [2, 3],

               [[4, 5],
                [6, 7]]])
```

```
In [32]: a3.size    # It has 8 items - like shape: 2,2,2 = 8
```

```
Out[32]: 8
```

```
In [33]: a2
```

```
Out[33]: array([[ 0.,  1.,  2.,  3.],
               [ 4.,  5.,  6.,  7.],
               [ 8.,  9., 10., 11.]])
```

```
In [34]: a2.size
```

```
Out[34]: 12
```

item size

```
In [35]: a1
```

```
Out[35]: array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
In [36]: a1.itemsize    # Bytes
```

```
Out[36]: 8
```

```
In [37]: a2.itemsize    # Integer 64 gives = 8 bytes
```

```
Out[37]: 8
```

```
In [38]: a3.itemsize    # Integer 64 gives = 8 bytes
```

```
Out[38]: 8
```

```
In [39]: a3
```

```
Out[39]: array([[0, 1],
               [2, 3],

               [[4, 5],
                [6, 7]]])
```

dtype

```
In [41]: print(a1.dtype)
         print(a2.dtype)
         print(a3.dtype)
```

```
int64
float64
int64
```

Changing Data Type

```
In [43]: # astype
         x = np.array([33, 22, 2.5])
         x
```

```
Out[43]: array([33. , 22. ,  2.5])
```

```
In [44]: x.astype(int)
```

```
Out[44]: array([33, 22,  2])
```

Array operations

```
In [46]: z1 = np.arange(12).reshape(3,4)
         z2 = np.arange(12,24).reshape(3,4)
```

```
In [47]: z1
```

```
Out[47]: array([[ 0,  1,  2,  3],
               [ 4,  5,  6,  7],
               [ 8,  9, 10, 11]])
```

```
In [48]: z2
```

```
Out[48]: array([[12, 13, 14, 15],
               [16, 17, 18, 19],
               [20, 21, 22, 23]])
```

Scalar operations

```
In [50]: # Arithmetic Operation (Addition)
```

```
z1 + 2
```

```
Out[50]: array([[ 2,  3,  4,  5],
               [ 6,  7,  8,  9],
               [10, 11, 12, 13]])
```

```
In [51]: # Subtraction
```

```
z1 - 2
```

```
Out[51]: array([[ -2, -1,  0,  1],
               [ 2,  3,  4,  5],
               [ 6,  7,  8,  9]])
```

```
In [52]: # Multiplication
```

```
z1 * 2
```

```
Out[52]: array([[ 0,  2,  4,  6],
               [ 8, 10, 12, 14],
               [16, 18, 20, 22]])
```

```
In [53]: # Power
```

```
z1 ** 2
```

```
Out[53]: array([[ 0,  1,  4,  9],
               [16, 25, 36, 49],
               [64, 81, 100, 121]])
```

```
In [54]: # Modulo
```

```
z1 % 2
```

```
Out[54]: array([[0, 1, 0, 1],
               [0, 1, 0, 1],
               [0, 1, 0, 1]])
```

Relational Operators

```
In [56]: z2
```

```
Out[56]: array([[12, 13, 14, 15],
               [16, 17, 18, 19],
               [20, 21, 22, 23]])
```

```
In [57]: z2 > 2 # If 2 is greater than everything gives True
```

```
Out[57]: array([[ True,  True,  True,  True],
               [ True,  True,  True,  True],
               [ True,  True,  True,  True]])
```

```
In [58]: z2 > 20
```

```
Out[58]: array([[False, False, False, False],
               [False, False, False, False],
               [False,  True,  True,  True]])
```

Vector Operation

```
In [60]: z1
```

```
Out[60]: array([[ 0,  1,  2,  3],
               [ 4,  5,  6,  7],
               [ 8,  9, 10, 11]])
```

```
In [61]: z2
```

```
Out[61]: array([[12, 13, 14, 15],
               [16, 17, 18, 19],
               [20, 21, 22, 23]])
```

```
In [62]: # Arithmetic
z1 + z2   # Both numpy array Shape is same, we can add item wise
```

```
Out[62]: array([[12, 14, 16, 18],
               [20, 22, 24, 26],
               [28, 30, 32, 34]])
```

```
In [63]: z1 * z2
```

```
Out[63]: array([[ 0, 13, 28, 45],
               [ 64, 85, 108, 133],
               [160, 189, 220, 253]])
```

```
In [64]: z1 - z2
```

```
Out[64]: array([[-12, -12, -12, -12],
               [-12, -12, -12, -12],
               [-12, -12, -12, -12]])
```

```
In [65]: z1 / z2
```

```
Out[65]: array([[0.         , 0.07692308, 0.14285714, 0.2         ],
               [0.25        , 0.29411765, 0.33333333, 0.36842105],
               [0.4         , 0.42857143, 0.45454545, 0.47826087]])
```

Array Functions

```
In [67]: k1 = np.random.random((3,3))
k1 = np.round(k1*100)
k1
```

```
Out[67]: array([[70.,  7., 83.],
               [ 8., 23., 70.],
               [92., 55., 85.]])
```

```
In [68]: # Max
np.max(k1)
```

```
Out[68]: np.float64(92.0)
```

```
In [69]: # Min
np.min(k1)
```

```
Out[69]: np.float64(7.0)
```

```
In [70]: # Sum
np.sum(k1)
```

```
Out[70]: np.float64(493.0)
```

```
In [71]: # Product ----> Multiplication
np.prod(k1)
```

```
Out[71]: np.float64(225299110960000.0)
```

In Numpy

0 = column, 1 = row

```
In [74]: # If we want maximum of every row
np.max(k1, axis = 1)
```

```
Out[74]: array([83., 70., 92.])
```

```
In [75]: # Maximum of every column
np.max(k1, axis = 0)
```

```
Out[75]: array([92., 55., 85.])
```

```
In [76]: # Product of every column
np.prod(k1, axis = 0)
```

```
Out[76]: array([ 51520.,  8855., 493850.])
```

Statistics related functions

```
In [78]: # Mean
k1
```

```
Out[78]: array([[70.,  7., 83.],
               [ 8., 23., 70.],
               [92., 55., 85.]])
```

```
In [79]: np.mean(k1)
```

```
Out[79]: np.float64(54.77777777777778)
```

```
In [80]: # Mean of every column
```

```
k1.mean(axis = 0)
```

```
Out[80]: array([56.66666667, 28.33333333, 79.33333333])
```

```
In [81]: # Median
```

```
np.median(k1)
```

```
Out[81]: np.float64(70.0)
```

```
In [82]: np.median(k1, axis = 1)
```

```
Out[82]: array([70., 23., 85.])
```

```
In [83]: # Standard deviation
```

```
np.std(k1)
```

```
Out[83]: np.float64(31.69219244117382)
```

```
In [84]: np.std(k1, axis = 0)
```

```
Out[84]: array([35.56527645, 19.95550606,  6.64997911])
```

```
In [85]: # Variance
```

```
np.var(k1)
```

```
Out[85]: np.float64(1004.3950617283949)
```

Trigonometry Functions

```
In [87]: np.sin(k1) # sin
```

```
Out[87]: array([[ 0.77389068,  0.6569866 ,  0.96836446],
               [ 0.98935825, -0.8462204 ,  0.77389068],
               [-0.77946607, -0.99975517, -0.17607562]])
```

```
In [88]: np.cos(k1)
```

```
Out[88]: array([[ 0.6333192 ,  0.75390225,  0.24954012],
               [-0.14550003, -0.53283302,  0.6333192 ],
               [-0.62644445,  0.02212676, -0.98437664]])
```

```
In [89]: np.tan(k1)
```

```
Out[89]: array([[ 1.22195992,  0.87144798,  3.88059631],
               [-6.79971146,  1.58815308,  1.22195992],
               [ 1.24427006, -45.18308791,  0.17887017]])
```

dot product

```
In [91]: s2 = np.arange(12).reshape(3,4)
        s3 = np.arange(12,24).reshape(4,3)
```

```
In [92]: s2
```

```
Out[92]: array([[ 0,  1,  2,  3],
               [ 4,  5,  6,  7],
               [ 8,  9, 10, 11]])
```

```
In [93]: s3
```

```
Out[93]: array([[12, 13, 14],
               [15, 16, 17],
               [18, 19, 20],
               [21, 22, 23]])
```

```
In [94]: np.dot(s2,s3)  # dot product of s2 , s3
```

```
Out[94]: array([[114, 120, 126],
               [378, 400, 422],
               [642, 680, 718]])
```

Log and Exponents

```
In [96]: np.exp(s2)
```

```
Out[96]: array([[1.00000000e+00, 2.71828183e+00, 7.38905610e+00, 2.00855369e+01],
               [5.45981500e+01, 1.48413159e+02, 4.03428793e+02, 1.09663316e+03],
               [2.98095799e+03, 8.10308393e+03, 2.20264658e+04, 5.98741417e+04]])
```

round / floor / ceil

(i) round

```
In [99]: # Round to the nearest integer
        arr = np.array([1.2, 2.7, 3.5, 4.9])
        rounded_arr = np.round(arr)
        print(rounded_arr)
```

```
[1.  3.  4.  5.]
```

```
In [100]: # Round to two decimals
        arr = np.array([1.234, 2.567, 3.891])
        rounded_arr = np.round(arr, decimals = 2)
        print(rounded_arr)
```

```
[1.23 2.57 3.89]
```

```
In [101... # Randomly
np.round(np.random.random((2,3))*100)
```

```
Out[101... array([[ 3., 89., 80.],
        [82., 26., 79.]])
```

(ii) floor

```
In [103... # Floor operation
arr = np.array([1.2, 2.7, 3.5, 4.9])
floored_arr = np.floor(arr)
print(floored_arr)
```

```
[1. 2. 3. 4.]
```

```
In [106... np.floor(np.random.random((2,3))*100) # Gives the smallest integer. Ex: 6.8 = 6
```

```
Out[106... array([[72., 18., 49.],
        [43., 26., 47.]])
```

(iii) Ceil

```
In [105... arr = np.array([1.2, 2.7, 3.5, 4.9])
ceiled_arr = np.ceil(arr)
print(ceiled_arr)
```

```
[2. 3. 4. 5.]
```

```
In [107... np.ceil(np.random.random((2,3))*100) # Gives highest integer. Ex: 7.8 = 8
```

```
Out[107... array([[57., 45., 83.],
        [44., 37., 79.]])
```

Indexing and Slicing

```
In [108... p1 = np.arange(10)
p2 = np.arange(12).reshape(3,4)
p3 = np.arange(8).reshape(2,2,2)
```

```
In [109... p1
```

```
Out[109... array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
In [110... p2
```

```
Out[110... array([[ 0,  1,  2,  3],
        [ 4,  5,  6,  7],
        [ 8,  9, 10, 11]])
```

```
In [111... p3
```

```
Out[111... array([[ [0, 1],
        [2, 3]],

        [[4, 5],
        [6, 7]]])
```

Indexing on 1D array

```
In [112...] p1  
Out[112...] array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
In [113...] # Fetching Last item  
  
p1[-1]
```

```
Out[113...] np.int64(9)
```

```
In [114...] # Fetching first item  
  
p1[0]
```

```
Out[114...] np.int64(0)
```

Indexing 2D array

```
In [115...] p2  
Out[115...] array([[ 0,  1,  2,  3],  
                [ 4,  5,  6,  7],  
                [ 8,  9, 10, 11]])
```

```
In [116...] # Fetching desired element : 6  
  
p2[1,2] # Here, 1st index row (second row), 2nd index column (3rd column) -- i
```

```
Out[116...] np.int64(6)
```

```
In [117...] # Fetching desired element : 11  
  
p2[2,3] # row = 2, column = 3
```

```
Out[117...] np.int64(11)
```

```
In [118...] # Fetching desired element  
p2[1,0] # row = 1, column = 0
```

```
Out[118...] np.int64(4)
```

Indexing on 3D (Tensors)

```
In [119...] p3  
Out[119...] array([[[0, 1],  
                  [2, 3]],  
                  [[4, 5],  
                  [6, 7]]])
```

```
In [120...] # Fetching desired element : 5
```

```
p3[1,0,1]
```

```
Out[120...] np.int64(5)
```

```
In [ ]: # Here 3D is consists of 2, 2D array, so Firstly we take 1 because our desired  
# is 5 is in second matrix which is 1 and 1 row so 0 and second column so 1.
```

```
In [121...] # Fetching desired element : 2
```

```
p3[0,1,0]
```

```
Out[121...] np.int64(2)
```

```
In [ ]: # Here 3D is consists of 2, 2D array, so Firstly we take 0 because our desired  
# is 2 is in first matrix which is 0 and 2 row so 1 and first column so 0.
```

```
In [122...] # Fetching desired element : 0
```

```
p3[0,0,0]
```

```
Out[122...] np.int64(0)
```

```
In [ ]: # Here 3D is consists of 2, 2D array, so Firstly we take 0 because our desired  
# is 0 is in first matrix which is 0 and 1 row so 0 and first column so 0.
```

```
In [123...] # Fetching desired element : 6
```

```
p3[1,1,0]
```

```
Out[123...] np.int64(6)
```

```
In [ ]: # Here 3D is consists of 2, 2D array, so Firstly we take 0 because our desired  
# is 6 is in second matrix which is 1 and 2nd row so 1 and first column so 0.
```

Slicing

Slicing on 1D

```
In [124...] p1
```

```
Out[124...] array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
In [125...] # Fetching desired element are : 2, 3, 4
```

```
p1[2:5]
```

```
Out[125...] array([2, 3, 4])
```

```
In [126...] # Alternate (Same as Python)
```

```
p1[2:5:2]
```

```
Out[126...] array([2, 4])
```

Slicing on 2D

```
In [127... p2
```

```
Out[127... array([[ 0,  1,  2,  3],
         [ 4,  5,  6,  7],
         [ 8,  9, 10, 11]])
```

```
In [128... # Fetching total first row

p2[0, :]
```

```
Out[128... array([0, 1, 2, 3])
```

```
In [129... # Fetching total third column

p2[:, 2]
```

```
Out[129... array([ 2,  6, 10])
```

```
In [130... # Fetch 5, 6 and 9, 10

p2
```

```
Out[130... array([[ 0,  1,  2,  3],
         [ 4,  5,  6,  7],
         [ 8,  9, 10, 11]])
```

```
In [131... p2[1:3] # For rows
```

```
Out[131... array([[ 4,  5,  6,  7],
         [ 8,  9, 10, 11]])
```

```
In [132... p2[1:3, 1:3] # For columns
```

```
Out[132... array([[ 5,  6],
         [ 9, 10]])
```

```
In [133... # Fetch 0,3 and 8, 11

p2
```

```
Out[133... array([[ 0,  1,  2,  3],
         [ 4,  5,  6,  7],
         [ 8,  9, 10, 11]])
```

```
In [134... p2[:, :2, ::3]
```

```
Out[134... array([[ 0,  3],
         [ 8, 11]])
```

```
In [135... # Fetch 1,3 and 9,11

p2
```

```
Out[135... array([[ 0,  1,  2,  3],
         [ 4,  5,  6,  7],
         [ 8,  9, 10, 11]])
```

```
In [136... p2[:,2] # For rows
```

```
Out[136... array([[ 0,  1,  2,  3],  
        [ 8,  9, 10, 11]])
```

```
In [137... p2[:,2, 1::2] # Columns
```

```
Out[137... array([[ 1,  3],  
        [ 9, 11]])
```

```
In [138... # Fetch only 4, 7
```

```
p2
```

```
Out[138... array([[ 0,  1,  2,  3],  
        [ 4,  5,  6,  7],  
        [ 8,  9, 10, 11]])
```

```
In [139... p2[1] # First rows
```

```
Out[139... array([4, 5, 6, 7])
```

```
In [140... p2[1,::3] # Second columns
```

```
Out[140... array([4, 7])
```

```
In [141... # Fetch 1, 2, 3 and 5, 6, 7
```

```
p2
```

```
Out[141... array([[ 0,  1,  2,  3],  
        [ 4,  5,  6,  7],  
        [ 8,  9, 10, 11]])
```

```
In [142... p2[0:2] # First fetched rows
```

```
Out[142... array([[0, 1, 2, 3],  
        [4, 5, 6, 7]])
```

```
In [143... p2[0:2, 1: ] # For columns
```

```
Out[143... array([[1, 2, 3],  
        [5, 6, 7]])
```

```
In [144... # Fetch 1, 3 and 5, 7
```

```
p2
```

```
Out[144... array([[ 0,  1,  2,  3],  
        [ 4,  5,  6,  7],  
        [ 8,  9, 10, 11]])
```

```
In [145... p2[0:2] # For rows
```

```
Out[145... array([[0, 1, 2, 3],  
        [4, 5, 6, 7]])
```

```
In [146... p2[0:2, 1::2]
```

```
Out[146...] array([[1, 3],  
          [5, 7]])
```

Slicing in 3D

```
In [147...] p3 = np.arange(27).reshape(3,3,3)  
p3
```

```
Out[147...] array([[[ 0,  1,  2],  
                  [ 3,  4,  5],  
                  [ 6,  7,  8]],  
                  [[ 9, 10, 11],  
                  [12, 13, 14],  
                  [15, 16, 17]],  
                  [[18, 19, 20],  
                  [21, 22, 23],  
                  [24, 25, 26]]])
```

```
In [148...] # Fetch second matrix  
  
p3[1]
```

```
Out[148...] array([[ 9, 10, 11],  
                  [12, 13, 14],  
                  [15, 16, 17]])
```

```
In [149...] # Fetch first and last  
  
p3[:, :2]
```

```
Out[149...] array([[[ 0,  1,  2],  
                  [ 3,  4,  5],  
                  [ 6,  7,  8]],  
                  [[18, 19, 20],  
                  [21, 22, 23],  
                  [24, 25, 26]]])
```

```
In [150...] # Fetch 1, 2D array's 2 row ---> 3,4,5  
p3
```

```
Out[150...] array([[[ 0,  1,  2],  
                  [ 3,  4,  5],  
                  [ 6,  7,  8]],  
                  [[ 9, 10, 11],  
                  [12, 13, 14],  
                  [15, 16, 17]],  
                  [[18, 19, 20],  
                  [21, 22, 23],  
                  [24, 25, 26]]])
```

```
In [151...] p3[0] # First numpy array
```

```
Out[151...] array([[0, 1, 2],  
          [3, 4, 5],  
          [6, 7, 8]])
```

```
In [152...] p3[0,1,:]
```

```
Out[152...] array([3, 4, 5])
```

```
In [153...] # Fetch 2 numpy array, middle column --->10, 13, 16  
  
p3
```

```
Out[153...] array([[ 0,  1,  2],  
                  [ 3,  4,  5],  
                  [ 6,  7,  8]],  
  
               [[ 9, 10, 11],  
               [12, 13, 14],  
               [15, 16, 17]],  
  
               [[18, 19, 20],  
               [21, 22, 23],  
               [24, 25, 26]])
```

```
In [154...] p3[1] # Middle array
```

```
Out[154...] array([[ 9, 10, 11],  
                  [12, 13, 14],  
                  [15, 16, 17]])
```

```
In [155...] p3[1,:,1]
```

```
Out[155...] array([10, 13, 16])
```

```
In [156...] # Fetch 3 array ---> 22, 23, 25, 26  
  
p3
```

```
Out[156...] array([[[ 0,  1,  2],  
                  [ 3,  4,  5],  
                  [ 6,  7,  8]],  
  
                  [[ 9, 10, 11],  
                  [12, 13, 14],  
                  [15, 16, 17]],  
  
                  [[18, 19, 20],  
                  [21, 22, 23],  
                  [24, 25, 26]])
```

```
In [157...] p3[2] # Last row
```

```
Out[157...] array([[18, 19, 20],  
                  [21, 22, 23],  
                  [24, 25, 26]])
```

```
In [158...] p3[2, 1:] # Last two rows
```

```
Out[158... array([[21, 22, 23],  
          [24, 25, 26]])
```

```
In [159... p3[2, 1:,1:] # Last two columns
```

```
Out[159... array([[22, 23],  
          [25, 26]])
```

```
In [160... # Fetch 0, 2, 18, 20  
p3
```

```
Out[160... array([[ 0,  1,  2],  
          [ 3,  4,  5],  
          [ 6,  7,  8]],  
  
          [[ 9, 10, 11],  
          [12, 13, 14],  
          [15, 16, 17]],  
  
          [[18, 19, 20],  
          [21, 22, 23],  
          [24, 25, 26]]])
```

```
In [161... p3[0::2] # for arrays
```

```
Out[161... array([[ 0,  1,  2],  
          [ 3,  4,  5],  
          [ 6,  7,  8]],  
  
          [[18, 19, 20],  
          [21, 22, 23],  
          [24, 25, 26]]])
```

```
In [162... p3[0::2, 0] # For rows
```

```
Out[162... array([[ 0,  1,  2],  
          [18, 19, 20]])
```

```
In [163... p3[0::2, 0, ::2] # For columns
```

```
Out[163... array([[ 0,  2],  
          [18, 20]])
```

Iterating

```
In [164... p1
```

```
Out[164... array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
In [165... # Looping on 1D array
```

```
for i in p1:  
    print(i)
```

0
1
2
3
4
5
6
7
8
9

In [166... p2

Out[166... array([[0, 1, 2, 3],
 [4, 5, 6, 7],
 [8, 9, 10, 11]])

In [167... *# Looping on 2D array*

```
for i in p2:  
    print(i)    # Prints rows
```

[0 1 2 3]
[4 5 6 7]
[8 9 10 11]

In [168... p3

Out[168... array([[[0, 1, 2],
 [3, 4, 5],
 [6, 7, 8]],

 [[9, 10, 11],
 [12, 13, 14],
 [15, 16, 17]],

 [[18, 19, 20],
 [21, 22, 23],
 [24, 25, 26]]])

In [169...

```
for i in p3:  
    print(i)
```

[[0 1 2]
 [3 4 5]
 [6 7 8]]
[[9 10 11]
 [12 13 14]
 [15 16 17]]
[[18 19 20]
 [21 22 23]
 [24 25 26]]

In [170...

```
for i in np.nditer(p3):  
    print(i)
```

0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26

Reshaping

(i) Transpose

In [171...

p2

Out[171... array([[0, 1, 2, 3],
 [4, 5, 6, 7],
 [8, 9, 10, 11]])

In [172... np.transpose(p2)

Out[172... array([[0, 4, 8],
 [1, 5, 9],
 [2, 6, 10],
 [3, 7, 11]])

In [173... *# Another method*

p2.T

Out[173... array([[0, 4, 8],
 [1, 5, 9],
 [2, 6, 10],
 [3, 7, 11]])

In [174... p3

```
Out[174...] array([[ 0,  1,  2],
          [ 3,  4,  5],
          [ 6,  7,  8]],

        [[ 9, 10, 11],
          [12, 13, 14],
          [15, 16, 17]],

        [[18, 19, 20],
          [21, 22, 23],
          [24, 25, 26]])
```

```
In [175...] p3.T
```

```
Out[175...] array([[ 0,  9, 18],
          [ 3, 12, 21],
          [ 6, 15, 24]],

        [[ 1, 10, 19],
          [ 4, 13, 22],
          [ 7, 16, 25]],

        [[ 2, 11, 20],
          [ 5, 14, 23],
          [ 8, 17, 26]])
```

(ii) Ravel

```
In [176...] p2
```

```
Out[176...] array([[ 0,  1,  2,  3],
          [ 4,  5,  6,  7],
          [ 8,  9, 10, 11]])
```

```
In [177...] p2.ravel()
```

```
Out[177...] array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11])
```

```
In [178...] p3
```

```
Out[178...] array([[ 0,  1,  2],
          [ 3,  4,  5],
          [ 6,  7,  8]],

        [[ 9, 10, 11],
          [12, 13, 14],
          [15, 16, 17]],

        [[18, 19, 20],
          [21, 22, 23],
          [24, 25, 26]])
```

```
In [179...] p3.ravel()
```

```
Out[179...] array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16,
          17, 18, 19, 20, 21, 22, 23, 24, 25, 26])
```

Stacking

```
In [180... # Horizontal Stacking

w1 = np.arange(12).reshape(3,4)
w2 = np.arange(12,24).reshape(3,4)
```

```
In [181... w1
```

```
Out[181... array([[ 0,  1,  2,  3],
        [ 4,  5,  6,  7],
        [ 8,  9, 10, 11]])
```

```
In [182... w2
```

```
Out[182... array([[12, 13, 14, 15],
        [16, 17, 18, 19],
        [20, 21, 22, 23]])
```

```
In [183... np.hstack((w1,w2))
```

```
Out[183... array([[ 0,  1,  2,  3, 12, 13, 14, 15],
        [ 4,  5,  6,  7, 16, 17, 18, 19],
        [ 8,  9, 10, 11, 20, 21, 22, 23]])
```

```
In [184... # Vertical stacking
```

```
w1
```

```
Out[184... array([[ 0,  1,  2,  3],
        [ 4,  5,  6,  7],
        [ 8,  9, 10, 11]])
```

```
In [185... w2
```

```
Out[185... array([[12, 13, 14, 15],
        [16, 17, 18, 19],
        [20, 21, 22, 23]])
```

```
In [186... np.vstack((w1, w2))
```

```
Out[186... array([[ 0,  1,  2,  3],
        [ 4,  5,  6,  7],
        [ 8,  9, 10, 11],
        [12, 13, 14, 15],
        [16, 17, 18, 19],
        [20, 21, 22, 23]])
```

Splitting

```
In [187... # Horizontal Splitting
```

```
w1
```

```
Out[187... array([[ 0,  1,  2,  3],
        [ 4,  5,  6,  7],
        [ 8,  9, 10, 11]])
```

```
In [188... np.hsplit(w1,2) # Splitting by 2
```

```
Out[188...  [array([[0, 1],
          [4, 5],
          [8, 9]]),
          array([[ 2,  3],
                  [ 6,  7],
                  [10, 11]])]
```

```
In [189... np.hsplit(w1, 4) # Splitting by 4
```

```
Out[189...  [array([[0],
          [4],
          [8]]),
          array([[1],
                  [5],
                  [9]]),
          array([[ 2],
                  [ 6],
                  [10]]),
          array([[ 3],
                  [ 7],
                  [11]])]
```

```
In [190... # Vertical Splitting
w2
```

```
Out[190...  array([[12, 13, 14, 15],
          [16, 17, 18, 19],
          [20, 21, 22, 23]])
```

```
In [191... np.vsplit(w2,3) # Splitting into 3 rows
```

```
Out[191...  [array([[12, 13, 14, 15]]),
          array([[16, 17, 18, 19]]),
          array([[20, 21, 22, 23]])]
```

Speed of List VS NumPy

List

```
In [197... # Element-wise addition

a = [ i for i in range(10000000)]
b = [i for i in range(10000000,20000000)]

c = []

import time

start = time.time()
for i in range(len(a)):
    c.append(a[i] + b[i])

print(time.time()-start)
```

2.843620777130127

Numpy

```
In [198... import numpy as np

a = np.arange(10000000)
b = np.arange(10000000, 20000000)

start = time.time()
c = a+b
print(time.time()-start)
```

0.1257627010345459

Memory used for List VS Array

List

```
In [199... P = [i for i in range(10000000)]

import sys

sys.getsizeof(P)
```

Out[199... 89095160

Numpy

```
In [200... R = np.arange(10000000)

sys.getsizeof(R)
```

Out[200... 80000112

```
In [201... # We can decrease more in numpy
R = np.arange(10000000, dtype=np.int16)
sys.getsizeof(R)
```

Out[201... 20000112

Advance Slicing and Indexing

```
In [202... # Normal Indexing and slicing

w = np.arange(12).reshape(4,3)
w
```

Out[202... array([[0, 1, 2],
 [3, 4, 5],
 [6, 7, 8],
 [9, 10, 11]])

```
In [203... # Fetching 5 from array

w[1,2]
```

```
Out[203... np.int64(5)
```

```
In [204... # Fetching 4,5,7,8  
w[1:3]
```

```
Out[204... array([[3, 4, 5],  
        [6, 7, 8]])
```

```
In [205... w[1:3 , 1:3]
```

```
Out[205... array([[4, 5],  
        [7, 8]])
```

Fancy Indexing

```
In [206... w
```

```
Out[206... array([[ 0,  1,  2],  
        [ 3,  4,  5],  
        [ 6,  7,  8],  
        [ 9, 10, 11]])
```

```
In [207... # Fetch 1,3,4 row  
  
w[[0,2,3]]
```

```
Out[207... array([[ 0,  1,  2],  
        [ 6,  7,  8],  
        [ 9, 10, 11]])
```

```
In [208... # New array  
  
z = np.arange(24).reshape(6,4)  
z
```

```
Out[208... array([[ 0,  1,  2,  3],  
        [ 4,  5,  6,  7],  
        [ 8,  9, 10, 11],  
        [12, 13, 14, 15],  
        [16, 17, 18, 19],  
        [20, 21, 22, 23]])
```

```
In [209... # Fetch 1, 3, ,4, 6 rows  
  
z[[0,2,3,5]]
```

```
Out[209... array([[ 0,  1,  2,  3],  
        [ 8,  9, 10, 11],  
        [12, 13, 14, 15],  
        [20, 21, 22, 23]])
```

```
In [210... # Fetch 1,3,4 columns  
  
z[:,[0,2,3]]
```

```
Out[210...] array([[ 0,  2,  3],
        [ 4,  6,  7],
        [ 8, 10, 11],
        [12, 14, 15],
        [16, 18, 19],
        [20, 22, 23]])
```

Boolean Indexing

```
In [211...] G = np.random.randint(1,100,24).reshape(6,4)
```

```
In [212...] G
```

```
Out[212...] array([[ 5, 31, 41,  9],
        [89, 66, 52, 24],
        [75, 63, 21, 69],
        [61, 64, 24, 30],
        [80, 24, 89,  8],
        [94, 35, 58, 12]], dtype=int32)
```

```
In [213...] # Find all numbers greater than 50
```

```
G > 50
```

```
Out[213...] array([[False, False, False, False],
        [ True,  True,  True, False],
        [ True,  True, False,  True],
        [ True,  True, False, False],
        [ True, False,  True, False],
        [ True, False,  True, False]])
```

```
In [214...] # Where is True , it gives result , everything other that removed.we got value
```

```
G[G > 50]
```

```
Out[214...] array([89, 66, 52, 75, 63, 69, 61, 64, 80, 89, 94, 58], dtype=int32)
```

```
In [215...] # Find out even numbers
```

```
G % 2 == 0
```

```
Out[215...] array([[False, False, False, False],
        [False,  True,  True,  True],
        [False, False, False, False],
        [False,  True,  True,  True],
        [ True,  True, False,  True],
        [ True, False,  True,  True]])
```

```
In [216...] # Gives only the even numbers
```

```
G [ G % 2 == 0]
```

```
Out[216...] array([66, 52, 24, 64, 24, 30, 80, 24,  8, 94, 58, 12], dtype=int32)
```

```
In [217...] # Find all numbers greater than 50 and are even
```

```
(G > 50 ) & (G % 2 == 0)
```

```
Out[217...] array([[False, False, False, False],
        [False, True, True, False],
        [False, False, False, False],
        [False, True, False, False],
        [ True, False, False, False],
        [ True, False, True, False]])
```

```
In [218...] # Result
```

```
G [(G > 50) & (G % 2 == 0)]
```

```
Out[218...] array([66, 52, 64, 80, 94, 58], dtype=int32)
```

```
In [219...] # Find all numbers not divisible by 7
```

```
G % 7 == 0
```

```
Out[219...] array([[False, False, False, False],
        [False, False, False, False],
        [False, True, True, False],
        [False, False, False, False],
        [False, False, False, False],
        [False, True, False, False]])
```

```
In [220...] #Result
```

```
G[~(G % 7 == 0)] # (~) = Not
```

```
Out[220...] array([ 5, 31, 41,  9, 89, 66, 52, 24, 75, 69, 61, 64, 24, 30, 80, 24, 89,
        8, 94, 58, 12], dtype=int32)
```

Broadcasting

```
In [221...] # Same shape
```

```
a = np.arange(6).reshape(2,3)
b = np.arange(6,12).reshape(2,3)
```

```
print(a)
print(b)
```

```
print(a+b)
```

```
[[0 1 2]
 [3 4 5]]
[[ 6  7  8]
 [ 9 10 11]]
[[ 6  8 10]
 [12 14 16]]
```

```
In [222...] # Different shape
```

```
a = np.arange(6).reshape(2,3)
b = np.arange(3).reshape(1,3)
```

```
print(a)
print(b)
```

```
print(a+b)
```

```
[[0 1 2]
 [3 4 5]]
[[0 1 2]]
[[0 2 4]
 [3 5 7]]
```

```
In [224... # More examples

a = np.arange(12).reshape(4,3)
b = np.arange(3)

print(a)      # 2D
```

```
[[ 0  1  2]
 [ 3  4  5]
 [ 6  7  8]
 [ 9 10 11]]
```

```
In [225... print(b)      # 1D
```

```
[0 1 2]
```

```
In [226... print(a+b)      # Arithmetic Operation
```

```
[[ 0  2  4]
 [ 3  5  7]
 [ 6  8 10]
 [ 9 11 13]]
```

```
In [227... # Could not be Broadcast

a = np.arange(12).reshape(3,4)
b = np.arange(3)

print(a)
print(b)

print(a+b)
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
[0 1 2]
```

```
-----
ValueError                                Traceback (most recent call last)
Cell In[227], line 9
      5
      6 print(a)
      7 print(b)
      8
----> 9 print(a+b)

ValueError: operands could not be broadcast together with shapes (3,4) (3,)
```

```
In [228... a = np.arange(3).reshape(1,3)
b = np.arange(3).reshape(3,1)

print(a)
print(b)

print(a+b)
```

```
[[0 1 2]]
[[0]]
[1]
[2]]
[[0 1 2]
 [1 2 3]
 [2 3 4]]
```

In [229...

```
a = np.arange(3).reshape(1,3)
b = np.arange(4).reshape(4,1)

print(a)
print(b)

print(a + b)
```

```
[[0 1 2]]
[[0]]
[1]
[2]
[3]]
[[0 1 2]
 [1 2 3]
 [2 3 4]
 [3 4 5]]
```

In [230...

```
a = np.array([1])
# shape -> (1,1) stretched to 2,2

b = np.arange(4).reshape(2,2)
# shape -> (2,2)

print(a)
print(b)

print(a+b)
```

```
[1]
[[0 1]
 [2 3]]
[[1 2]
 [3 4]]
```

In [231...

```
# Doesn't work

a = np.arange(12).reshape(3,4)
b = np.arange(12).reshape(4,3)

print(a)
print(b)

print(a+b)
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
[[ 0  1  2]
 [ 3  4  5]
 [ 6  7  8]
 [ 9 10 11]]
```

```
-----
ValueError                                Traceback (most recent call last)
Cell In[231], line 9
      5
      6 print(a)
      7 print(b)
      8
----> 9 print(a+b)

ValueError: operands could not be broadcast together with shapes (3,4) (4,3)
```

```
In [232... # Not Working
a = np.arange(16).reshape(4,4)
b = np.arange(4).reshape(2,2)

print(a)
print(b)

print(a+b)
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]
 [12 13 14 15]]
[[0 1]
 [2 3]]
```

```
-----
ValueError                                Traceback (most recent call last)
Cell In[232], line 8
      4
      5 print(a)
      6 print(b)
      7
----> 8 print(a+b)

ValueError: operands could not be broadcast together with shapes (4,4) (2,2)
```

Working with Mathematical Formulas

```
In [233... k = np.arange(10)
```

```
In [234... k
```

```
Out[234... array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
In [235... np.sum(k)
```

```
Out[235... np.int64(45)
```

```
In [236... np.sin(k)
```

```
Out[236... array([ 0.          ,  0.84147098,  0.90929743,  0.14112001, -0.7568025 ,
        -0.95892427, -0.2794155 ,  0.6569866 ,  0.98935825,  0.41211849])
```

sigmoid

```
In [237... def sigmoid(array):  
    return 1/(1+np.exp(-(array)))  
  
k = np.arange(10)  
sigmoid(k)
```

```
Out[237... array([0.5, 0.73105858, 0.88079708, 0.95257413, 0.98201379,
        0.99330715, 0.99752738, 0.99908895, 0.99966465, 0.99987661])
```

```
In [238... k = np.arange(100)
sigmoid(k)
```

[illegible]

Mean squared error

```
In [239... actual = np.random.randint(1,50,25)
predicted = np.random.randint(1,50,25)
```

In [240...	actual
------------	--------

```
Out[240...] array([45,  1, 44, 45, 24, 30, 29, 26, 10, 32, 29, 22, 27,  1, 24,  1, 19,
        47, 31, 13, 46,  8, 40, 40, 36], dtype=int32)
```

In [241... predicted

```
Out[241... array([47,  9, 40, 39, 15, 21, 41, 14, 18, 31, 14, 25, 38, 42,  7, 35, 38,
        44, 40, 36, 19,  8, 34, 11, 16], dtype=int32)
```

```
In [242... def mse(actual,predicted):
            return np.mean((actual-predicted)**2)

mse(actual,predicted)
```

```
Out[242... np.float64(284.08)
```

```
In [243... # Detailed
          actual-predicted
```

```
Out[243...] array([ -2,  -8,   4,   6,   9,   9, -12,  12,  -8,   1,  15,  -3, -11,
        -41,  17, -34, -19,   3,  -9, -23,  27,   0,   6,  29,  20],
        dtype=int32)
```

```
In [244...] (actual-predicted)**2
```

```
Out[244...] array([   4,   64,   16,   36,   81,   81,  144,  144,   64,    1,  225,
         9,  121, 1681,  289, 1156,  361,    9,   81,  529,  729,    0,
        36,  841,  400], dtype=int32)
```

```
In [246...] np.mean((actual-predicted)**2)
```

```
Out[246...] np.float64(284.08)
```

Working with Missing Values

```
In [247...] # Working with missing values -> np.nan
```

```
S = np.array([1,2,3,4,np.nan,6])
S
```

```
Out[247...] array([ 1.,  2.,  3.,  4., nan,  6.])
```

```
In [248...] np.isnan(S)
```

```
Out[248...] array([False, False, False, False,  True, False])
```

```
In [249...] S[np.isnan(S)]    # Nan Values
```

```
Out[249...] array([nan])
```

```
In [250...] S[~np.isnan(S)]    # Non - Nan Values
```

```
Out[250...] array([1.,  2.,  3.,  4.,  6.])
```

Plotting Graphs

```
In [251...] # Plotting a 2D plot
```

```
# x = y
```

```
x = np.linspace(-10,10,100)
x
```

```
Out[251...] array([-10.          , -9.7979798 , -9.5959596 , -9.39393939,
        -9.19191919, -8.98989899, -8.78787879, -8.58585859,
        -8.38383838, -8.18181818, -7.97979798, -7.77777778,
        -7.57575758, -7.37373737, -7.17171717, -6.96969697,
        -6.76767677, -6.56565657, -6.36363636, -6.16161616,
        -5.95959596, -5.75757576, -5.55555556, -5.35353535,
        -5.15151515, -4.94949495, -4.74747475, -4.54545455,
        -4.34343434, -4.14141414, -3.93939394, -3.73737374,
        -3.53535354, -3.33333333, -3.13131313, -2.92929293,
        -2.72727273, -2.52525253, -2.32323232, -2.12121212,
        -1.91919192, -1.71717172, -1.51515152, -1.31313131,
        -1.11111111, -0.90909091, -0.70707071, -0.50505051,
        -0.3030303 , -0.1010101 ,  0.1010101 ,  0.3030303 ,
         0.50505051,  0.70707071,  0.90909091,  1.11111111,
         1.31313131,  1.51515152,  1.71717172,  1.91919192,
         2.12121212,  2.32323232,  2.52525253,  2.72727273,
         2.92929293,  3.13131313,  3.33333333,  3.53535354,
         3.73737374,  3.93939394,  4.14141414,  4.34343434,
         4.54545455,  4.74747475,  4.94949495,  5.15151515,
         5.35353535,  5.55555556,  5.75757576,  5.95959596,
         6.16161616,  6.36363636,  6.56565657,  6.76767677,
         6.96969697,  7.17171717,  7.37373737,  7.57575758,
         7.77777778,  7.97979798,  8.18181818,  8.38383838,
         8.58585859,  8.78787879,  8.98989899,  9.19191919,
         9.39393939,  9.5959596 ,  9.7979798 , 10.          ])
```

```
In [252...] y = x
```

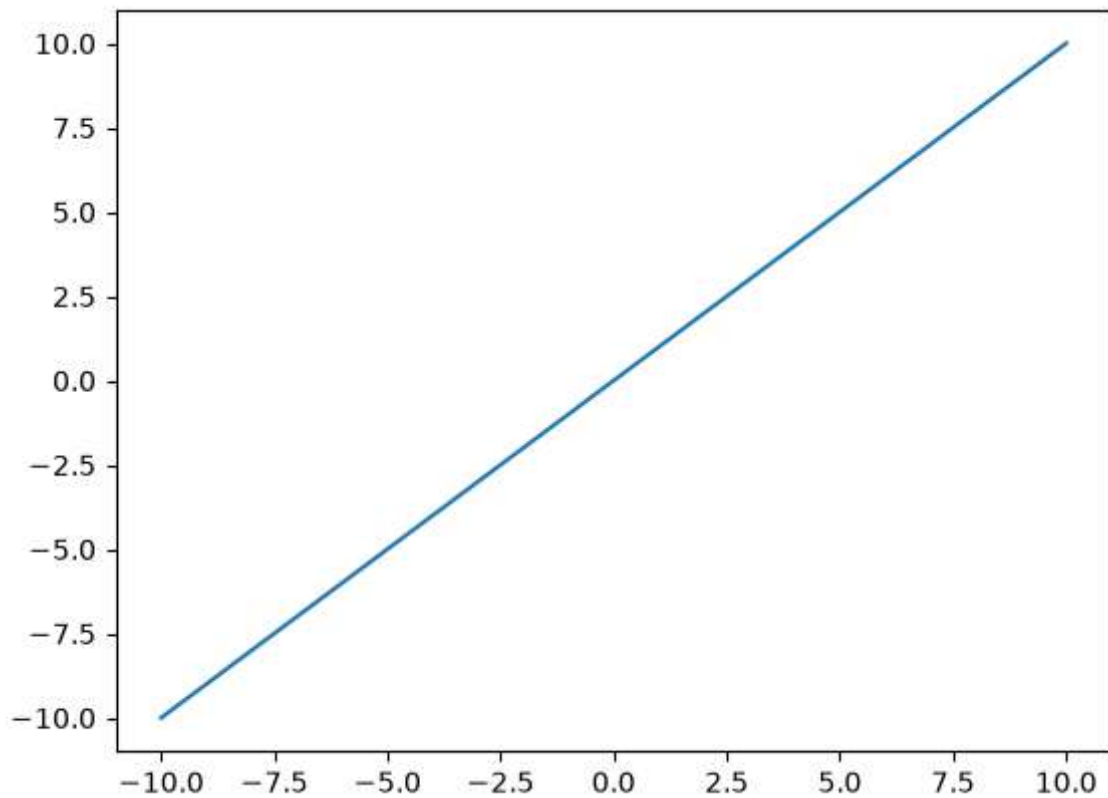
```
In [253...] y
```

```
Out[253...] array([-10.          , -9.7979798 , -9.5959596 , -9.39393939,
        -9.19191919, -8.98989899, -8.78787879, -8.58585859,
        -8.38383838, -8.18181818, -7.97979798, -7.77777778,
        -7.57575758, -7.37373737, -7.17171717, -6.96969697,
        -6.76767677, -6.56565657, -6.36363636, -6.16161616,
        -5.95959596, -5.75757576, -5.55555556, -5.35353535,
        -5.15151515, -4.94949495, -4.74747475, -4.54545455,
        -4.34343434, -4.14141414, -3.93939394, -3.73737374,
        -3.53535354, -3.33333333, -3.13131313, -2.92929293,
        -2.72727273, -2.52525253, -2.32323232, -2.12121212,
        -1.91919192, -1.71717172, -1.51515152, -1.31313131,
        -1.11111111, -0.90909091, -0.70707071, -0.50505051,
        -0.3030303 , -0.1010101 ,  0.1010101 ,  0.3030303 ,
         0.50505051,  0.70707071,  0.90909091,  1.11111111,
         1.31313131,  1.51515152,  1.71717172,  1.91919192,
         2.12121212,  2.32323232,  2.52525253,  2.72727273,
         2.92929293,  3.13131313,  3.33333333,  3.53535354,
         3.73737374,  3.93939394,  4.14141414,  4.34343434,
         4.54545455,  4.74747475,  4.94949495,  5.15151515,
         5.35353535,  5.55555556,  5.75757576,  5.95959596,
         6.16161616,  6.36363636,  6.56565657,  6.76767677,
         6.96969697,  7.17171717,  7.37373737,  7.57575758,
         7.77777778,  7.97979798,  8.18181818,  8.38383838,
         8.58585859,  8.78787879,  8.98989899,  9.19191919,
         9.39393939,  9.5959596 ,  9.7979798 , 10.          ])
```

```
In [254...] import matplotlib.pyplot as plt

plt.plot(x ,y)
```

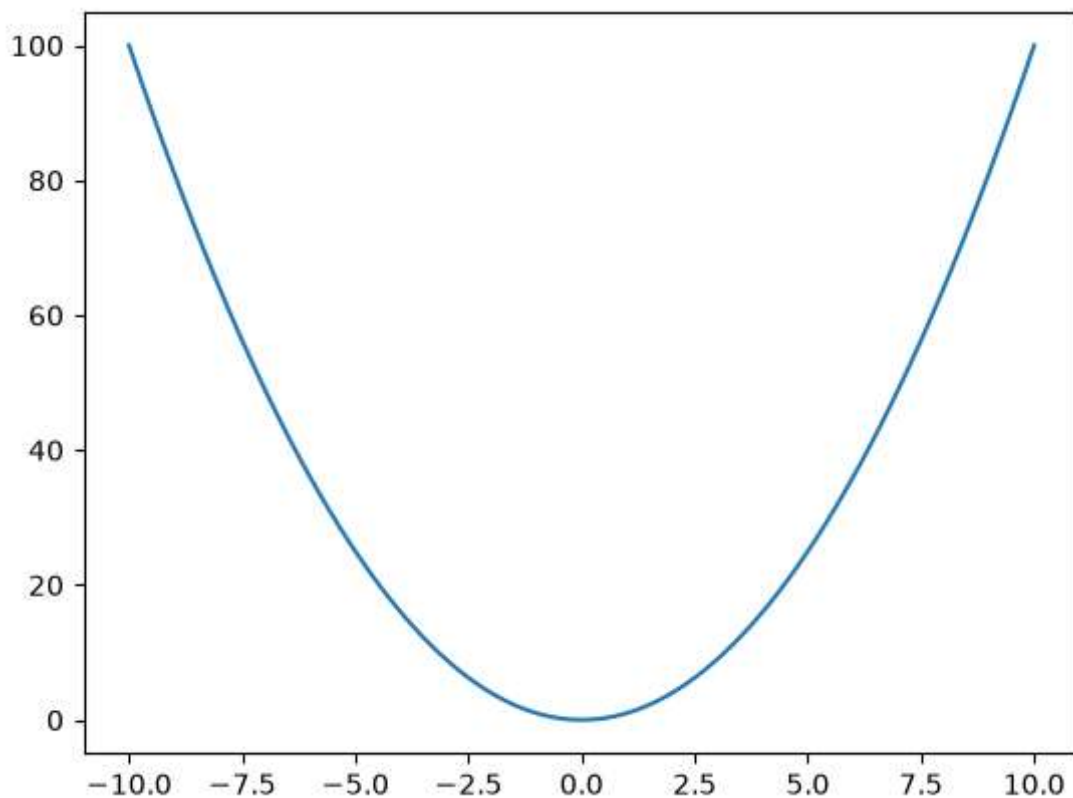
Out[254...] [<matplotlib.lines.Line2D at 0x22b019b0d70>]



In [255...

```
# y = x^2
x = np.linspace(-10,10,100)
y = x**2
plt.plot(x,y)
```

Out[255...] [<matplotlib.lines.Line2D at 0x22b019eb4d0>]

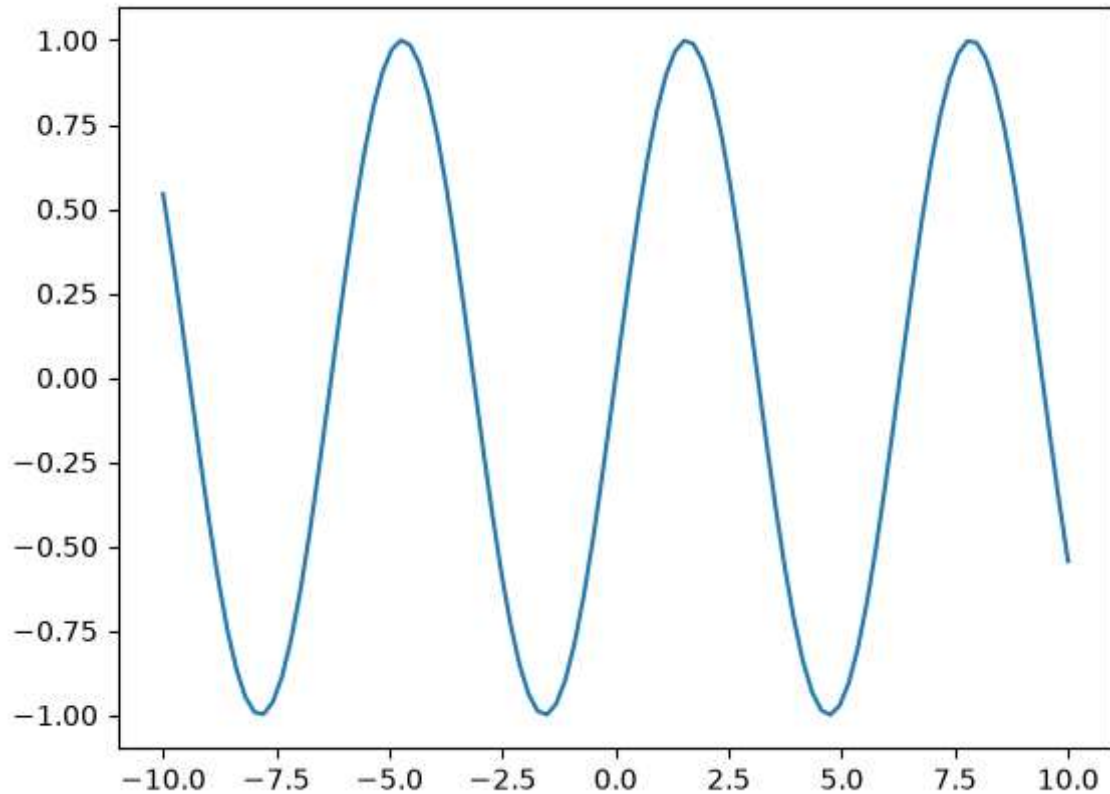


```
In [256... # y = sin(x)

x = np.linspace(-10,10,100)
y = np.sin(x)

plt.plot(x,y)
```

Out[256... [<matplotlib.lines.Line2D at 0x22b03be0ad0>]



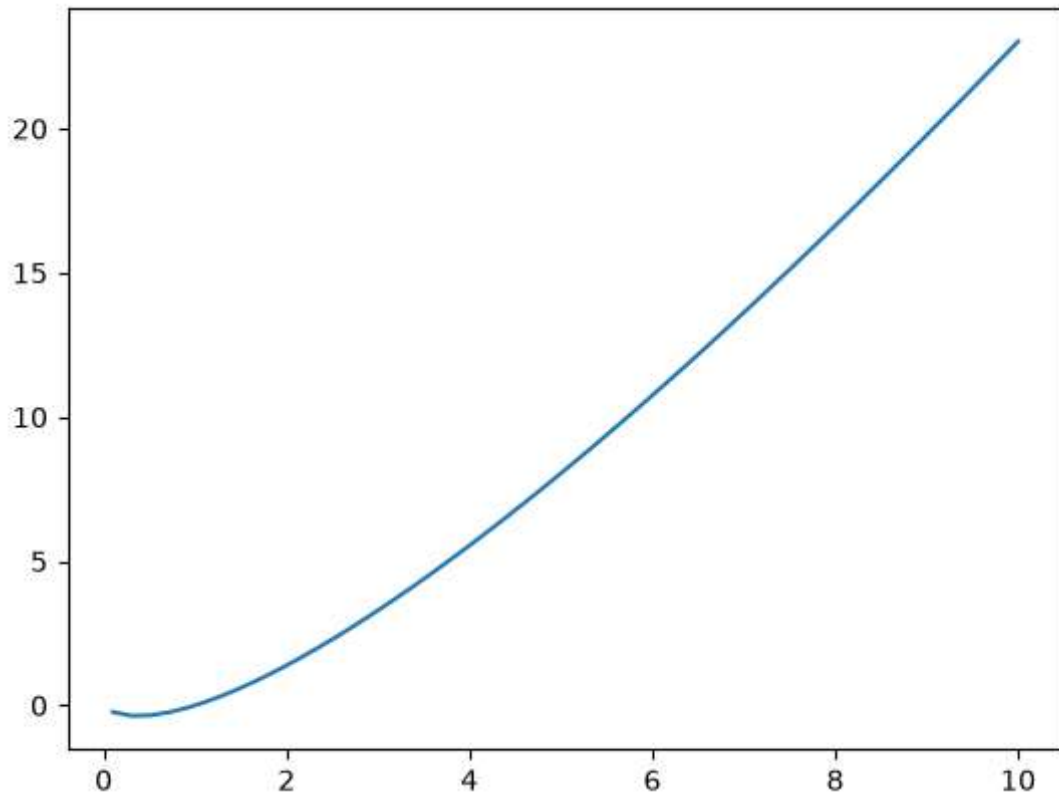
```
In [257... # y = xlog(x)

x = np.linspace(-10,10,100)
y = x * np.log(x)

plt.plot(x,y)
```

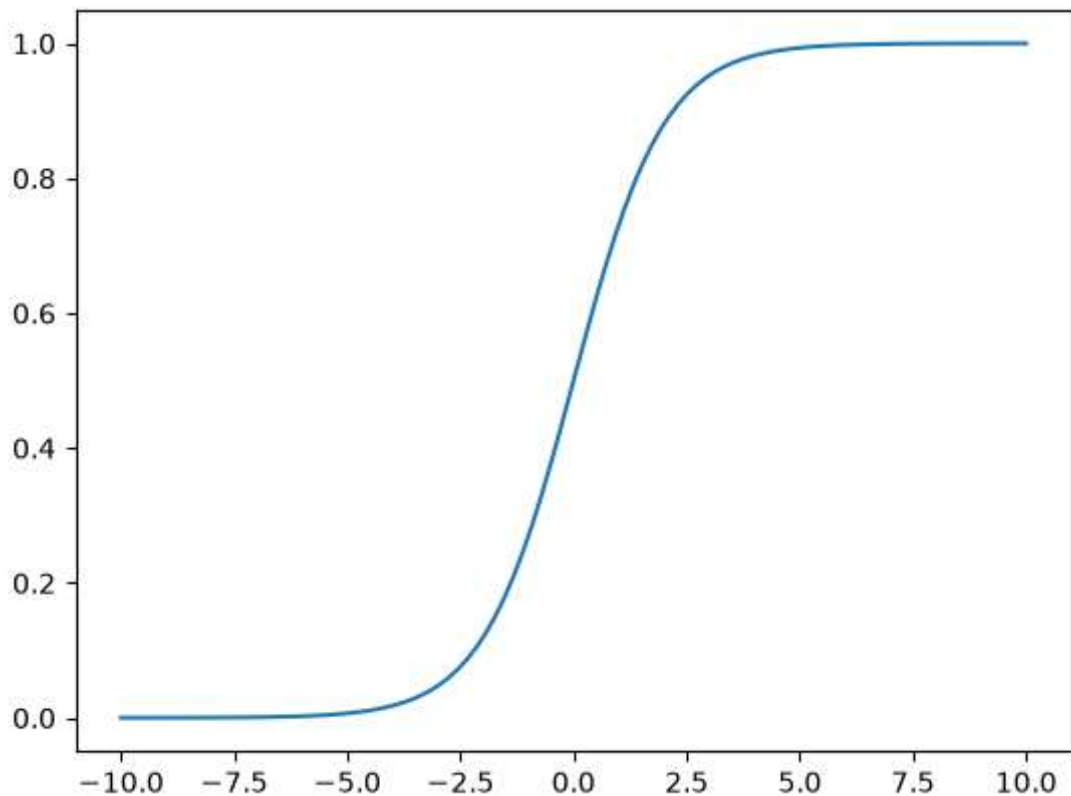
C:\Users\DELL\AppData\Local\Temp\ipykernel_6436\217697418.py:4: RuntimeWarning: invalid value encountered in log
y = x * np.log(x)

Out[257... [<matplotlib.lines.Line2D at 0x22b03c16ba0>]



```
In [258... # Sigmoid  
  
x = np.linspace(-10,10,100)  
y = 1/(1+np.exp(-x))  
  
plt.plot(x,y)
```

```
Out[258... [<matplotlib.lines.Line2D at 0x22b03c570e0>]
```



```
In [259... import numpy as np
import matplotlib.pyplot as plt
```

Meshgrid

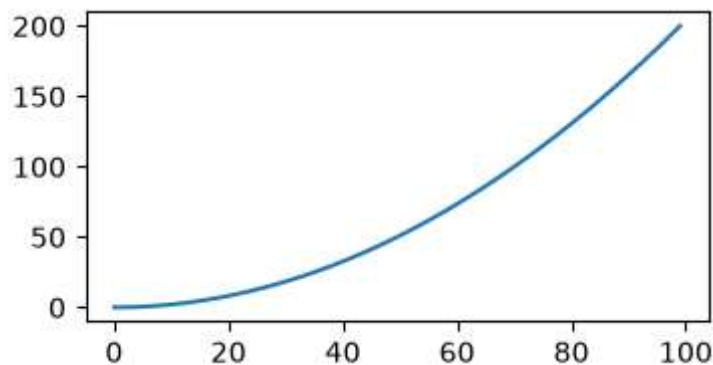
```
In [260... x = np.linspace(0,10,100)
y = np.linspace(0,10,100)
```

Try to create 2D function

```
In [261... f = x**2+y**2
```

```
In [ ]: ### Plot
```

```
In [262... plt.figure(figsize=(4,2))
plt.plot(f)
plt.show()
```



```
In [263... x = np.arange(3)
y = np.arange(3)
```

```
In [264... x
```

```
Out[264... array([0, 1, 2])
```

```
In [265... y
```

```
Out[265... array([0, 1, 2])
```

Generating a meshgrid:

```
In [266... xv, yv = np.meshgrid(x,y)
```

```
In [267... xv
```

```
Out[267... array([[0, 1, 2],
        [0, 1, 2],
        [0, 1, 2]])
```

```
In [268... yv
```

```
Out[268... array([[0, 0, 0],
        [1, 1, 1],
        [2, 2, 2]])
```

```
In [269... P = np.linspace(-4, 4, 9)
V = np.linspace(-5, 5, 11)
print(P)
print(V)
```

```
[-4. -3. -2. -1.  0.  1.  2.  3.  4.]
[-5. -4. -3. -2. -1.  0.  1.  2.  3.  4.  5.]
```

```
In [270... P_1, V_1 = np.meshgrid(P,V)
```

```
In [271... print(P_1)
```

```
[-4. -3. -2. -1.  0.  1.  2.  3.  4.]
[-4. -3. -2. -1.  0.  1.  2.  3.  4.]
[-4. -3. -2. -1.  0.  1.  2.  3.  4.]
[-4. -3. -2. -1.  0.  1.  2.  3.  4.]
[-4. -3. -2. -1.  0.  1.  2.  3.  4.]
[-4. -3. -2. -1.  0.  1.  2.  3.  4.]
[-4. -3. -2. -1.  0.  1.  2.  3.  4.]
[-4. -3. -2. -1.  0.  1.  2.  3.  4.]
[-4. -3. -2. -1.  0.  1.  2.  3.  4.]
[-4. -3. -2. -1.  0.  1.  2.  3.  4.]
[-4. -3. -2. -1.  0.  1.  2.  3.  4.]
```

```
In [272... print(V_1)
```

```
[-5. -5. -5. -5. -5. -5. -5. -5. -5.]
[-4. -4. -4. -4. -4. -4. -4. -4. -4.]
[-3. -3. -3. -3. -3. -3. -3. -3. -3.]
[-2. -2. -2. -2. -2. -2. -2. -2. -2.]
[-1. -1. -1. -1. -1. -1. -1. -1. -1.]
[ 0.  0.  0.  0.  0.  0.  0.  0.  0.]
[ 1.  1.  1.  1.  1.  1.  1.  1.  1.]
[ 2.  2.  2.  2.  2.  2.  2.  2.  2.]
[ 3.  3.  3.  3.  3.  3.  3.  3.  3.]
[ 4.  4.  4.  4.  4.  4.  4.  4.  4.]
[ 5.  5.  5.  5.  5.  5.  5.  5.  5.]
```

Numpy Meshgrid Creates Coordinates for a Grid System

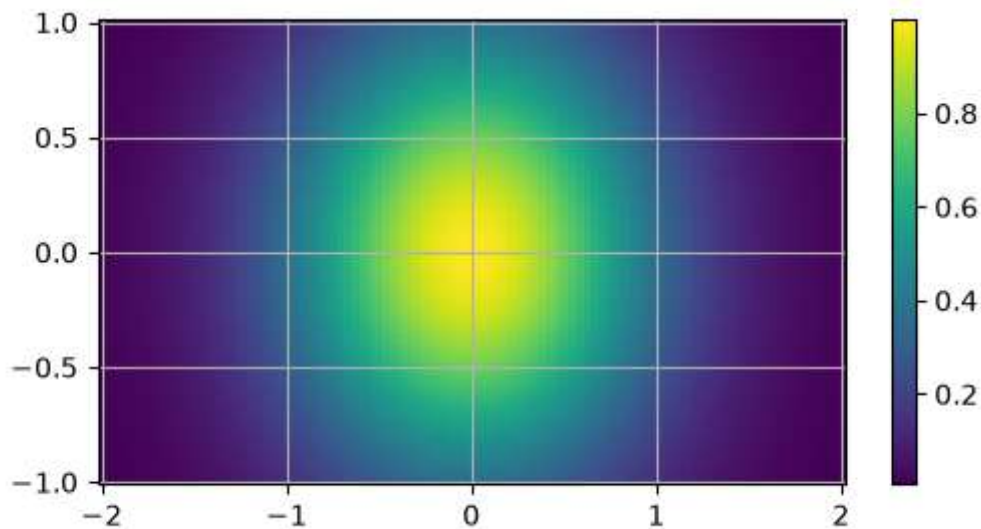
```
In [273... xv**2 + yv**2
```

```
Out[273... array([[0, 1, 4],
        [1, 2, 5],
        [4, 5, 8]])
```

```
In [274... x = np.linspace(-2,2,100)
y = np.linspace(-1,1,100)
xv, yv = np.meshgrid(x, y)
f = np.exp(-xv**2-yv**2)
```

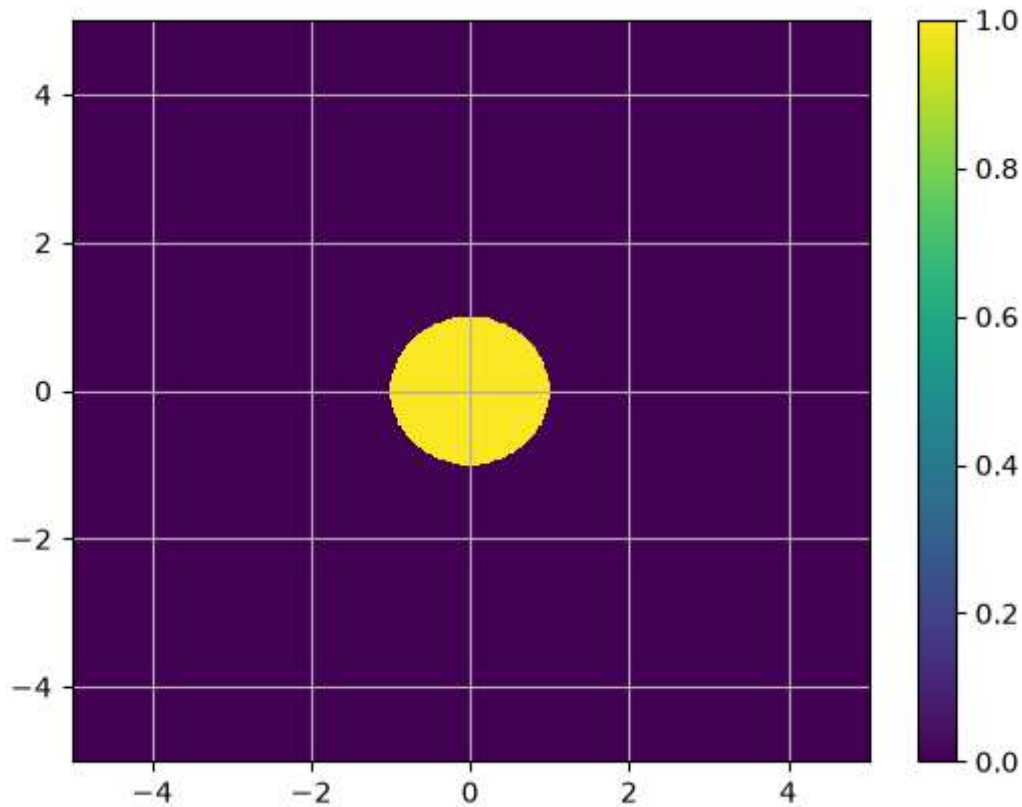
```
In [275... plt.figure(figsize=(6, 3))
plt.pcolormesh(xv, yv, f, shading='auto')
plt.colorbar()
```

```
plt.grid()  
plt.show()
```



In [277...

```
import numpy as np  
import matplotlib.pyplot as plt  
  
def f(x, y):  
    return np.where((x**2 + y**2 < 1), 1.0, 0.0)  
  
x = np.linspace(-5, 5, 500)  
y = np.linspace(-5, 5, 500)  
xv, yv = np.meshgrid(x, y)  
rectangular_mask = f(xv, yv)  
  
plt.pcolormesh(xv, yv, rectangular_mask, shading='auto')  
plt.colorbar()  
plt.grid()  
plt.show()
```



```
In [278... # numpy.linspace creates an array of  
# 9 linearly placed elements between  
# -4 and 4, both inclusive
```

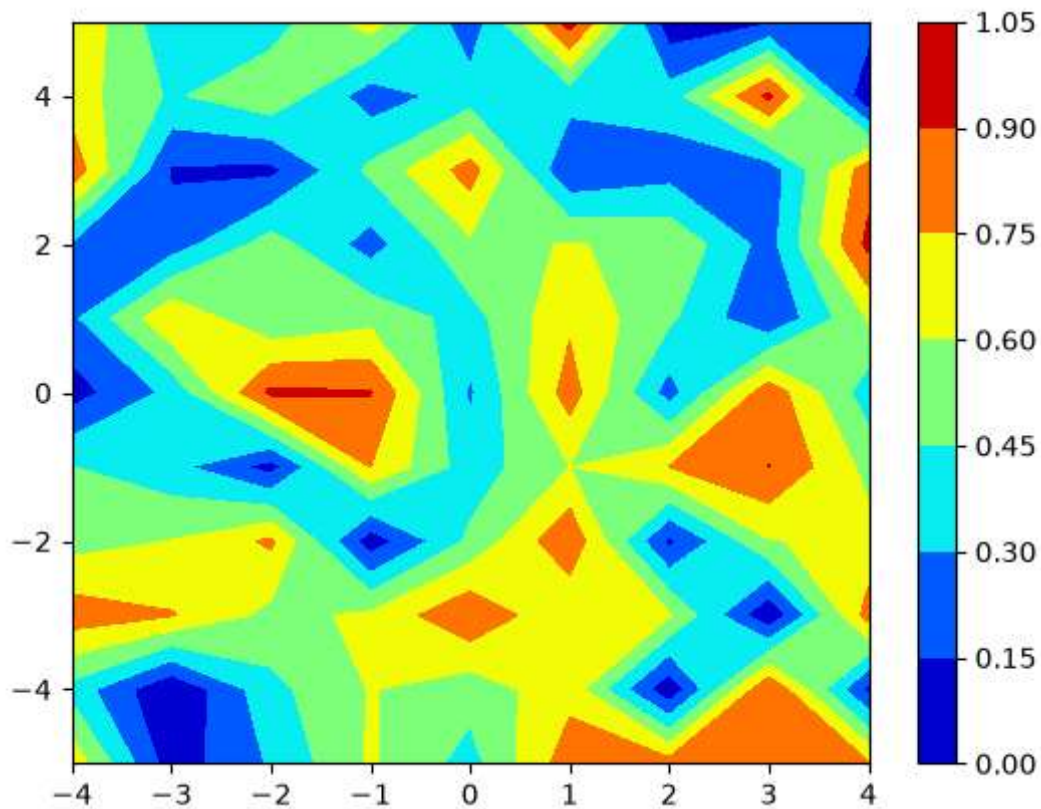
```
x = np.linspace(-4, 4, 9)
```

```
In [279... # numpy.linspace creates an array of  
# 9 linearly placed elements between  
# -4 and 4, both inclusive
```

```
y = np.linspace(-5, 5, 11)
```

```
In [280... x_1, y_1 = np.meshgrid(x, y)
```

```
In [281... random_data = np.random.random((11, 9))  
plt.contourf(x_1, y_1, random_data, cmap = 'jet')  
  
plt.colorbar()  
plt.show()
```

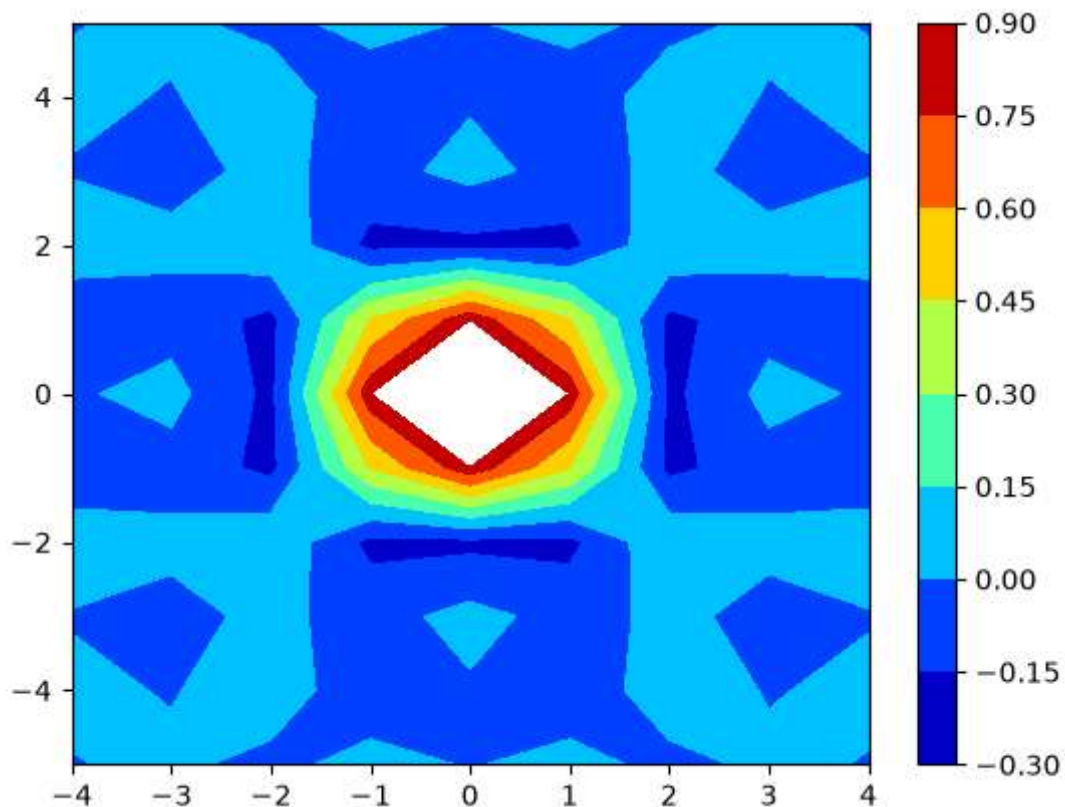


```
In [282... sine = (np.sin(x_1**2 + y_1**2))/(x_1**2 + y_1**2)
plt.contourf(x_1, y_1, sine, cmap = 'jet')

plt.colorbar()
plt.show()
```

C:\Users\DELL\AppData\Local\Temp\ipykernel_6436\2872376425.py:1: RuntimeWarning: invalid value encountered in divide

```
sine = (np.sin(x_1**2 + y_1**2))/(x_1**2 + y_1**2)
```



```
In [283... x_1, y_1 = np.meshgrid(x, y, sparse = True)
```

```
In [285... x_1
```

```
Out[285... array([[ -4.,  -3.,  -2.,  -1.,   0.,   1.,   2.,   3.,   4.]])
```

```
In [286... y_1
```

```
Out[286... array([[ -5.],  
        [ -4.],  
        [ -3.],  
        [ -2.],  
        [ -1.],  
        [  0.],  
        [  1.],  
        [  2.],  
        [  3.],  
        [  4.],  
        [  5.]])
```

np.sort

```
In [288... a = np.random.randint(1,100,15)    # 1D  
a
```

```
Out[288... array([97, 10, 11, 85, 86, 73, 22, 82, 67, 14,  2,  6, 79, 21, 87],  
        dtype=int32)
```

```
In [289... b = np.random.randint(1,100,24).reshape(6,4)    # 2D  
b
```

```
Out[289... array([[ 6, 63, 51, 44],  
        [68, 50, 66, 43],  
        [53, 31, 80, 49],  
        [98, 72, 62, 15],  
        [28, 72,  8, 84],  
        [66, 90, 85,  5]], dtype=int32)
```

```
In [290... np.sort(a)    # Default= Ascending
```

```
Out[290... array([ 2,  6, 10, 11, 14, 21, 22, 67, 73, 79, 82, 85, 86, 87, 97],  
        dtype=int32)
```

```
In [291... np.sort(a)[::-1]    # Descending order
```

```
Out[291... array([97, 87, 86, 85, 82, 79, 73, 67, 22, 21, 14, 11, 10,  6,  2],  
        dtype=int32)
```

```
In [292... np.sort(b)    # Row wise sorting
```

```
Out[292... array([[ 6, 44, 51, 63],  
        [43, 50, 66, 68],  
        [31, 49, 53, 80],  
        [15, 62, 72, 98],  
        [ 8, 28, 72, 84],  
        [ 5, 66, 85, 90]], dtype=int32)
```

```
In [293... np.sort(b,axis = 0)    # Column wise sorting
```

```
Out[293...] array([[ 6, 31,  8,  5],
        [28, 50, 51, 15],
        [53, 63, 62, 43],
        [66, 72, 66, 44],
        [68, 72, 80, 49],
        [98, 90, 85, 84]], dtype=int32)
```

np.append

```
In [294...] # Code
```

```
a
```

```
Out[294...] array([97, 10, 11, 85, 86, 73, 22, 82, 67, 14,  2,  6, 79, 21, 87],
        dtype=int32)
```

```
In [295...] np.append(a, 200)
```

```
Out[295...] array([ 97,  10,  11,  85,  86,  73,  22,  82,  67,  14,   2,   6,  79,
        21,  87, 200])
```

```
In [296...] b      # On 2D
```

```
Out[296...] array([[ 6, 63, 51, 44],
        [68, 50, 66, 43],
        [53, 31, 80, 49],
        [98, 72, 62, 15],
        [28, 72,  8, 84],
        [66, 90, 85,  5]], dtype=int32)
```

```
In [297...] # Adding Extra column :1
```

```
np.append(b,np.ones((b.shape[0],1)))
```

```
Out[297...] array([ 6., 63., 51., 44., 68., 50., 66., 43., 53., 31., 80., 49., 98.,
        72., 62., 15., 28., 72.,  8., 84., 66., 90., 85.,  5.,  1.,  1.,
        1.,  1.,  1.,  1.])
```

```
In [298...] np.append(b,np.ones((b.shape[0],1)),axis=1)
```

```
Out[298...] array([[ 6., 63., 51., 44.,  1.],
        [68., 50., 66., 43.,  1.],
        [53., 31., 80., 49.,  1.],
        [98., 72., 62., 15.,  1.],
        [28., 72.,  8., 84.,  1.],
        [66., 90., 85.,  5.,  1.]])
```

```
In [299...] # Adding random numbers in new column
```

```
np.append(b,np.random.random((b.shape[0],1)),axis=1)
```

```
Out[299...] array([[6.00000000e+00, 6.30000000e+01, 5.10000000e+01, 4.40000000e+01,
        1.87030896e-01],
        [6.80000000e+01, 5.00000000e+01, 6.60000000e+01, 4.30000000e+01,
        8.72039253e-01],
        [5.30000000e+01, 3.10000000e+01, 8.00000000e+01, 4.90000000e+01,
        2.68052480e-01],
        [9.80000000e+01, 7.20000000e+01, 6.20000000e+01, 1.50000000e+01,
        5.48496619e-01],
        [2.80000000e+01, 7.20000000e+01, 8.00000000e+00, 8.40000000e+01,
        4.39584746e-02],
        [6.60000000e+01, 9.00000000e+01, 8.50000000e+01, 5.00000000e+00,
        3.89800799e-01]])
```

np.concatenate

In [300...

```
# Code

c = np.arange(6).reshape(2,3)
d = np.arange(6,12).reshape(2,3)
```

In [301...

c

Out[301...

```
array([[0, 1, 2],
       [3, 4, 5]])
```

In [302...

d

Out[302...

```
array([[ 6,  7,  8],
       [ 9, 10, 11]])
```

In [303...

```
np.concatenate((c,d)) # Row wise
```

Out[303...

```
array([[ 0,  1,  2],
       [ 3,  4,  5],
       [ 6,  7,  8],
       [ 9, 10, 11]])
```

In [304...

```
np.concatenate((c,d)) # Column wise
```

Out[304...

```
array([[ 0,  1,  2],
       [ 3,  4,  5],
       [ 6,  7,  8],
       [ 9, 10, 11]])
```

np.unique

In [305...

```
# Code

e = np.array([1,1,2,2,3,3,4,4,5,5,6,6])
```

In [306...

e

Out[306...

```
array([1, 1, 2, 2, 3, 3, 4, 4, 5, 5, 6, 6])
```

In [308...

```
np.unique(e)
```

Out[308...

```
array([1, 2, 3, 4, 5, 6])
```

np.expand_dims

In [309...

```
# Code  
a
```

Out[309...

```
array([97, 10, 11, 85, 86, 73, 22, 82, 67, 14,  2,  6, 79, 21, 87],  
      dtype=int32)
```

In [310...

```
a.shape    # 1D
```

Out[310...

```
(15,)
```

In [311...

```
# Converting into 2D array  
  
np.expand_dims(a,axis = 0)
```

Out[311...

```
array([[97, 10, 11, 85, 86, 73, 22, 82, 67, 14,  2,  6, 79, 21, 87]],  
      dtype=int32)
```

In [312...

```
np.expand_dims(a,axis = 0).shape    # 2D
```

Out[312...

```
(1, 15)
```

In [313...

```
np.expand_dims(a,axis = 1)
```

Out[313...

```
array([[97],  
       [10],  
       [11],  
       [85],  
       [86],  
       [73],  
       [22],  
       [82],  
       [67],  
       [14],  
       [ 2],  
       [ 6],  
       [79],  
       [21],  
       [87]], dtype=int32)
```

In [314...

```
np.expand_dims(a,axis = 1).shape
```

Out[314...

```
(15, 1)
```

np.where

In [315...

```
a
```

Out[315...

```
array([97, 10, 11, 85, 86, 73, 22, 82, 67, 14,  2,  6, 79, 21, 87],  
      dtype=int32)
```

In [316...

```
# Find all indices with value greater than 50  
  
np.where(a>50)
```

```
Out[316... (array([ 0,  3,  4,  5,  7,  8, 12, 14]),)
```

```
In [317... # Replace all values > 50 with 0  
  
np.where(a>50,0,a)
```

```
Out[317... array([ 0, 10, 11,  0,  0,  0, 22,  0,  0, 14,  2,  6,  0, 21,  0],  
      dtype=int32)
```

```
In [318... # Print and replace all even numbers to  
  
np.where(a%2 == 0,0,a)
```

```
Out[318... array([97,  0, 11, 85,  0, 73,  0,  0, 67,  0,  0,  0, 79, 21, 87],  
      dtype=int32)
```

np.argmax

```
In [319... # Code  
a
```

```
Out[319... array([97, 10, 11, 85, 86, 73, 22, 82, 67, 14,  2,  6, 79, 21, 87],  
      dtype=int32)
```

```
In [320... np.argmax(a)
```

```
Out[320... np.int64(0)
```

```
In [321... b # On 2D
```

```
Out[321... array([[ 6, 63, 51, 44],  
      [68, 50, 66, 43],  
      [53, 31, 80, 49],  
      [98, 72, 62, 15],  
      [28, 72,  8, 84],  
      [66, 90, 85,  5]], dtype=int32)
```

```
In [322... np.argmax(b,axis =1) # Row wise biggest number : index
```

```
Out[322... array([1, 0, 2, 0, 3, 1])
```

```
In [323... np.argmax(b,axis =0) # Column wise biggest number : index
```

```
Out[323... array([3, 5, 5, 4])
```

np.argmin

```
In [324... a
```

```
Out[324... array([97, 10, 11, 85, 86, 73, 22, 82, 67, 14,  2,  6, 79, 21, 87],  
      dtype=int32)
```

```
In [325... np.argmin(a)
```

```
Out[325... np.int64(10)
```

On Statistics

np.cumsum

In [326...

a

Out[326...

```
array([97, 10, 11, 85, 86, 73, 22, 82, 67, 14,  2,  6, 79, 21, 87],
      dtype=int32)
```

In [327...

```
np.cumsum(a)
```

Out[327...

```
array([ 97, 107, 118, 203, 289, 362, 384, 466, 533, 547, 549, 555, 634,
       655, 742])
```

In [328...

b

Out[328...

```
array([[ 6, 63, 51, 44],
       [68, 50, 66, 43],
       [53, 31, 80, 49],
       [98, 72, 62, 15],
       [28, 72,  8, 84],
       [66, 90, 85,  5]], dtype=int32)
```

In [329...

```
np.cumsum(b)
```

Out[329...

```
array([[ 6,  69, 120, 164, 232, 282, 348, 391, 444, 475, 555,
        604, 702, 774, 836, 851, 879, 951, 959, 1043, 1109, 1199,
        1284, 1289])
```

In [330...

```
np.cumsum(b,axis=1)    # Row wise calculation or cumulative sum
```

Out[330...

```
array([[ 6,  69, 120, 164],
       [ 68, 118, 184, 227],
       [ 53,  84, 164, 213],
       [ 98, 170, 232, 247],
       [ 28, 100, 108, 192],
       [ 66, 156, 241, 246]])
```

In [331...

```
np.cumsum(b,axis=0)    # Column wise calculation or cumulative sum
```

Out[331...

```
array([[ 6,  63,  51,  44],
       [ 74, 113, 117,  87],
       [127, 144, 197, 136],
       [225, 216, 259, 151],
       [253, 288, 267, 235],
       [319, 378, 352, 240]])
```

In [332...

```
# np.cumprod --> Multiply
```

a

Out[332...

```
array([97, 10, 11, 85, 86, 73, 22, 82, 67, 14,  2,  6, 79, 21, 87],
      dtype=int32)
```

In [333...

```
np.cumprod(a)
```

```
Out[333...] array([          97,          970,         10670,
          906950,         77997700,        5693832100,
        125264306200,        10271673108400,        688202098262800,
          9634829375679200,        19269658751358400,        115617952508150400,
          9133818248143881600,        7342742473925997440, -6817447348272529280])
```

np.percentile

```
In [334...] a
```

```
Out[334...] array([97, 10, 11, 85, 86, 73, 22, 82, 67, 14,  2,  6, 79, 21, 87],
      dtype=int32)
```

```
In [335...] np.percentile(a,100)  # Max
```

```
Out[335...] np.float64(97.0)
```

```
In [336...] np.percentile(a,0)  # Min
```

```
Out[336...] np.float64(2.0)
```

```
In [337...] np.percentile(a,50)  # Median
```

```
Out[337...] np.float64(67.0)
```

```
In [338...] np.median(a)
```

```
Out[338...] np.float64(67.0)
```

np.histogram

```
In [339...] a
```

```
Out[339...] array([97, 10, 11, 85, 86, 73, 22, 82, 67, 14,  2,  6, 79, 21, 87],
      dtype=int32)
```

```
In [340...] np.histogram(a , bins= [10,20,30,40,50,60,70,80,90,100])
```

```
Out[340...] (array([3, 2, 0, 0, 0, 1, 2, 4, 1]),
      array([ 10,  20,  30,  40,  50,  60,  70,  80,  90, 100]))
```

```
In [341...] np.histogram(a , bins= [0,50,100])
```

```
Out[341...] (array([7, 8]), array([ 0,  50, 100]))
```

np.corrcoef

```
In [342...] salary = np.array([20000,40000,25000,35000,60000])
      experience = np.array([1,3,2,4,2])
```

```
In [343...] salary
```

```
Out[343...] array([20000, 40000, 25000, 35000, 60000])
```

```
In [344...] experience
```

```
Out[344...] array([1, 3, 2, 4, 2])
```

```
In [345...] np.corrcoef(salary,experience) # Correlation Coefficient
```

```
Out[345...] array([[1.          , 0.25344572],  
          [0.25344572, 1.          ]])
```

Utility Functions

(i) np.isin

```
In [346...] # Code  
a
```

```
Out[346...] array([97, 10, 11, 85, 86, 73, 22, 82, 67, 14,  2,  6, 79, 21, 87],  
          dtype=int32)
```

```
In [347...] items = [10,20,30,40,50,60,70,80,90,100]  
  
np.isin(a,items)
```

```
Out[347...] array([False,  True, False, False, False, False, False, False, False,  
          False, False, False, False, False, False])
```

```
In [348...] a[np.isin(a,items)]
```

```
Out[348...] array([10], dtype=int32)
```

(ii) np.flip

```
In [349...] # Code  
a
```

```
Out[349...] array([97, 10, 11, 85, 86, 73, 22, 82, 67, 14,  2,  6, 79, 21, 87],  
          dtype=int32)
```

```
In [350...] np.flip(a) # Reverse
```

```
Out[350...] array([87, 21, 79,  6,  2, 14, 67, 82, 22, 73, 86, 85, 11, 10, 97],  
          dtype=int32)
```

```
In [351...] b
```

```
Out[351...] array([[ 6, 63, 51, 44],  
          [68, 50, 66, 43],  
          [53, 31, 80, 49],  
          [98, 72, 62, 15],  
          [28, 72,  8, 84],  
          [66, 90, 85,  5]], dtype=int32)
```

```
In [353...] np.flip(b)
```

```
Out[353...] array([[ 5, 85, 90, 66],
        [84,  8, 72, 28],
        [15, 62, 72, 98],
        [49, 80, 31, 53],
        [43, 66, 50, 68],
        [44, 51, 63,  6]], dtype=int32)
```

```
In [354...] np.flip(b,axis = 1)  # Row
```

```
Out[354...] array([[44, 51, 63,  6],
        [43, 66, 50, 68],
        [49, 80, 31, 53],
        [15, 62, 72, 98],
        [84,  8, 72, 28],
        [ 5, 85, 90, 66]], dtype=int32)
```

```
In [355...] np.flip(b,axis = 0 )  # Column
```

```
Out[355...] array([[66, 90, 85,  5],
        [28, 72,  8, 84],
        [98, 72, 62, 15],
        [53, 31, 80, 49],
        [68, 50, 66, 43],
        [ 6, 63, 51, 44]], dtype=int32)
```

(iii) np.put

```
In [356...] # Code
a
```

```
Out[356...] array([97, 10, 11, 85, 86, 73, 22, 82, 67, 14,  2,  6, 79, 21, 87],
        dtype=int32)
```

```
In [357...] np.put(a,[0,1],[110,530])  # Permanent changes
```

```
In [358...] a
```

```
Out[358...] array([110, 530,  11,  85,  86,  73,  22,  82,  67,  14,   2,   6,  79,
        21,  87], dtype=int32)
```

(iv) np.delete

```
In [359...] # Code
a
```

```
Out[359...] array([110, 530,  11,  85,  86,  73,  22,  82,  67,  14,   2,   6,  79,
        21,  87], dtype=int32)
```

```
In [360...] np.delete(a,0)  # Deleted 0 index item
```

```
Out[360...] array([530,  11,  85,  86,  73,  22,  82,  67,  14,   2,   6,  79, 21,
        87], dtype=int32)
```

```
In [361...] np.delete(a,[0,2,4])  # Deleted 0,2,4 index items
```

```
Out[361...] array([530, 85, 73, 22, 82, 67, 14, 2, 6, 79, 21, 87],  
          dtype=int32)
```

Set Functions

```
In [362...] m = np.array([1,2,3,4,5])  
          n = np.array([3,4,5,6,7])
```

```
In [363...] # Union  
  
          np.union1d(m,n)
```

```
Out[363...] array([1, 2, 3, 4, 5, 6, 7])
```

```
In [365...] # Intersection  
  
          np.intersect1d(m,n)
```

```
Out[365...] array([3, 4, 5])
```

```
In [366...] # Set difference  
  
          np.setdiff1d(m,n)
```

```
Out[366...] array([1, 2])
```

```
In [367...] np.setdiff1d(n,m)
```

```
Out[367...] array([6, 7])
```

```
In [368...] # Set XOR  
  
          np.setxor1d(m,n)
```

```
Out[368...] array([1, 2, 6, 7])
```

```
In [382...] #in 1D ( Like membership operator)  
  
          np.isin(m,1)
```

```
Out[382...] array([ True, False, False, False, False])
```

```
In [383...] m[np.isin(m,1)]
```

```
Out[383...] array([1])
```

```
In [384...] np.isin(m,10)
```

```
Out[384...] array([False, False, False, False, False])
```

np.clip

```
In [375...] # code  
          a
```

```
Out[375...] array([110, 530, 11, 85, 86, 73, 22, 82, 67, 14, 2, 6, 79,
      21, 87], dtype=int32)
```

```
In [376...] np.clip(a, a_min=15 , a_max =50)
```

```
Out[376...] array([50, 50, 15, 50, 50, 50, 22, 50, 50, 15, 15, 15, 50, 21, 50],
      dtype=int32)
```

np.swapaxes

```
In [377...] arr = np.array([[1, 2, 3], [4, 5, 6]])
      swapped_arr = np.swapaxes(arr, 0, 1)
```

```
In [378...] arr
```

```
Out[378...] array([[1, 2, 3],
      [4, 5, 6]])
```

```
In [379...] swapped_arr
```

```
Out[379...] array([[1, 4],
      [2, 5],
      [3, 6]])
```

```
In [380...] print("Original array:")
      print(arr)
```

Original array:

```
[[1 2 3]
 [4 5 6]]
```

```
In [381...] print("Swapped array:")
      print(swapped_arr)
```

Swapped array:

```
[[1 4]
 [2 5]
 [3 6]]
```

```
In [ ]:
```